

Distributed DAG Task Scheduler: Revised Specification (MS2)

This revises our approved MS1 specification. The intended semester claims (quoted in each section) are unchanged. Under each one we state what the MS2 prototype actually guarantees today. Supporting tests and measurements are in the [MS2 progress report](#). Design detail is in the [architecture](#), and the wire format is in [api.md](#).

1. Overview

We are building a fault-tolerant distributed task scheduler for a video-processing platform. Each uploaded video creates a directed acyclic graph (DAG) of tasks such as transcoding, thumbnail generation, subtitle generation, and publishing. Some tasks can run in parallel, while others must wait for their dependencies. The system consists of one scheduler and multiple worker nodes. The scheduler tracks dependencies and assigns ready tasks to available workers. Workers execute tasks and report completion. If a worker fails before finishing, the unfinished task can be reassigned. We are not building a full video-sharing platform, authentication, billing, or production-quality video-processing algorithms.

In MS2 the system has four parts:

- **Scheduler.** A single Java 21 process makes every job, task and attempt decision under one mutex and keeps its state in memory.
- **Artifact store.** An HTTP service holds immutable files. Workers exchange inputs and outputs only through it.
- **Workers.** One-slot workers poll the scheduler and run a fixed allowlist of operations: small arithmetic fixtures, ffprobe, FFmpeg transcodes and thumbnails.
- **Client.** A Java command-line client uploads files, submits jobs, checks status and fetches results.

The video DAG (inspect → 720p, 360p, thumbnail and subtitles in parallel → publish) runs end to end on a committed synthetic clip. "Publish" writes a manifest of the produced files. The subtitles are a fixture file, not real transcription.

2. System model

We use the course's network, node and timing vocabulary.

- **Network:** fair-loss point-to-point links. Messages can be lost, delayed, duplicated or reordered; endpoints are honest. Every request that changes state carries a stable ID (job, claim or attempt identity), so a retried request has no extra effect.
- **Workers:** crash-recovery. A worker may stop at any time, permanently or not. A restarted worker is a new session with no memory of its previous attempt.
- **Scheduler:** assumed correct for the life of a run. A restart starts a new, empty run with a new run ID, and messages addressed to an older run are rejected. We claim no scheduler crash tolerance.
- **Artifact store:** assumed correct and available during a run. Stored objects never change.

- **Timing:** partially synchronous. Safety never depends on timing or synchronized clocks. Progress assumes messages are eventually delivered and operations finish within their limits. Durations are measured on a single process's monotonic clock. Ordering across machines comes from IDs and scheduler acknowledgments, never from comparing wall clocks.

3. Safety

A task will not be marked complete unless successful completion is reported, and a task will not run before its required dependencies finish. Retried tasks will not be recorded as multiple successful completions.

MS2 status: all three hold within a scheduler run.

- **No completion without a reported success.** A task becomes SUCCEEDED only on a SUCCESS report from the worker that currently owns its running attempt. Every declared output must already be published with the matching length and hash. A timeout, a failure, silence, or a file that merely exists never completes a task.
- **No task before its dependencies.** A task is offered to workers only after every parent has an accepted success. A worker starts executing only after the scheduler acknowledges its start, and its inputs are the parents' accepted outputs.
- **No duplicate completion.** A task keeps the same identity across retries, SUCCEEDED is final, and a replayed report gets the stored receipt back. A success arriving late from an older attempt is rejected.

These properties are checked automatically on the recorded event histories of service-backed tests, and the checker can also be run offline on saved evidence. Independent Step 13 verification recorded all 40 generated evidence directories as valid; see [VERIFICATION.md](#) and the [claim-to-test matrix](#). We assume workers are honest: the scheduler verifies identity, state and output integrity, but does not recompute results.

4. Liveness

If workers remain available, every ready task will eventually be assigned and either complete or be retried after failure. If a worker fails while executing a task, that unfinished task will eventually become available for reassignment.

MS2 status: the first claim holds partially; the second is intentionally not met.

Ready tasks are assigned in FIFO order. A task whose worker reports a failure goes to the back of the queue and is reassigned with a higher attempt number. There is no retry limit, so a job is never marked failed. This holds while workers stay up.

If a worker crashes while holding a task, MS2 has no lease, heartbeat or timeout that would take the task back. The task stays RUNNING under the dead worker forever, and everything downstream stays blocked. This is the one correctness violation MS2 demonstrates on purpose. The test `test_worker_crash_reassignment` demands reassignment and fails with `RecoveryNotObserved`. Step 13 independently observed exactly this failure under `make fault-demo`; `make test` recorded 86 passes, exactly one intentional XFAIL and no failures. Its exact fault-demo shell exit code was not preserved. The stuck task and job stay RUNNING, and downstream Y stays BLOCKED. Both

intended liveness claims are constrained by the same missing reclamation mechanism; its implementation is deferred to MS3.

No progress is promised during a permanent partition, while all workers are down, or under unbounded load.

5. Fault model

The system will handle worker crashes, worker restarts, temporary worker unavailability, and delayed or lost communication. Byzantine or malicious worker behavior is outside the scope of the project. These failures will be tested by stopping workers and introducing communication problems during active jobs.

MS2 tests worker stops and lost or duplicated messages. Broader communication-fault testing is planned with MS3's recovery work.

Fault	MS2 behavior
Lost, delayed or duplicated messages	Handled. Retries reuse the same IDs and get cached replies. A lost acknowledgment causes the report to be resent, never the work to be redone.
Worker restart	Safe. The new session cannot act on the old attempt. The old attempt is not recovered.
Worker crash holding a task	Safe but not live: the task stays stuck (Section 4).
Stale or conflicting reports	Rejected with no state change. The worker drops that attempt and keeps working.
Scheduler restart	New empty run. Messages from the old run get 409 STALE_RUN, and workers join the new run.
Artifact store unavailable	Submissions get a temporary 503, and nothing is accepted.
Byzantine workers	Out of scope.

6. Performance

We will measure job completion time, task throughput, scheduling latency, and recovery time after worker failures. Performance will be evaluated by submitting increasing numbers of concurrent DAG-based jobs under normal operation and while intentionally introducing worker failures.

MS2 status: three of the four metrics are measured under normal operation. We ran 1, 4 and 16 concurrent jobs against 1, 2 and 4 workers, five fresh repetitions each, with every service capped at 1 CPU and 512 MiB. All 45 runs completed.

Throughput is set by the number of workers: about 4.2 tasks/s per worker, reaching 16.7 tasks/s with 4 workers. An idle worker receives a ready task within about 5 ms. Results are in progress-report Section E and [results/handoffs/step-12/full-20261003c/](#). Step 13 separately completed a one-repetition c1-w1 Compose reproduction; it does not replace the measured matrix.

Recovery time cannot be measured yet, because MS2 does not recover from crashes. The crash test's 10-second observation window is a test limit, not a recovery measurement.

7. Scope

Implemented in MS2:

- DAG submission with validation and size limits;
- dependency tracking and parallel branches;
- retries after reported failures, with duplicate- and stale-report handling;
- the immutable artifact store;
- event histories and an offline invariant checker;
- the functional and video workloads;
- Docker Compose deployment;
- the benchmark;
- a Hokea deployment adapter for the course cluster.

The functional and video demo outputs were independently verified in Step 13: `result=22`, H.264 MP4s at 1280x720 and 640x360, and a PNG at 1280x720. The [Step 13 record](#) reports no implementation changes.

The Hokea adapter is verified locally. A run on the course cluster was attempted, but it was blocked before our services started because the course runner has a different Hokea version than the one we pin. This remains an external environment limitation, documented in [results/handoffs/step-11/FINAL-RESULTS.md](#); it is not a successful cluster workload or the intentional recovery XFAIL.

Deferred: recovering tasks from crashed workers (MS3); scheduler durability, replication and failover; artifact-store recovery; exactly-once external effects; authentication, UI, cancellation, priorities and retry limits; performance targets and recovery-time measurement.

8. MS3 direction

The approved MS3 direction is scheduler-side leases on the existing monotonic clock; these are not current MS2 guarantees. When a lease expires, the attempt is marked expired and the task returns to the ready queue. The existing owner and attempt checks already reject late results from an expired attempt, and per-attempt output paths already stop it from overwriting newer results. Once leases exist, the crash test's expected-failure marker is removed and the same test must pass.